

Kunskapskontroll 2, Deep Learning, Katarina Persson

Teoretiska frågor

1) Artificiell intelligens, AI enligt definition, en maskin som imiterar intelligens, eller en människa, eller andra levande organismers beteendemönster. Området består av ett antal underkategorier, som tex maskininlärning, där modellen lär sig utan explicit programmering, genom träning på data. Ytterligare avancerade modeller inom Maskininlärning finns i kategorin Deep Learning där modellens lärande och lösning sker genom olika lager, inspirerade av hjärnas struktur

Relation mellan dem: AI är den övergripande modellen, där maskininlärning är en huvudmetoderna, och mer avancerade modeller finns i underkategorin Deep Learning neurala nätverk.

2) Keras är ett Deep learning API, som används för att bygga, träna och utvärdera olika neurala nätverk. Sambandet är att Keras hanterar träningsprocesser och nätverksarkitektur, och TensorFlow utför de matematiska beräkningarna för nätverket och exekveringen av modellen.

3) En parameter, en modellvariabel som kan uppskattas genom att anpassa modellen efter givet data. Exempel på en parameter är linjens lutning och intercept, och används för att göra prediktioner.

Hyperparametrar däremot, är olika externa värden kopplade till modellen, som kan justeras i syfte att få modellen att prestera optimalt. Exempel på hyperparameter är antal lager, samt antal neuroner per lager, syfte är att optimera modellens funktionalitet.

4) När man skall göra modellval och modellutvärdering så kan man använda ett tränings, validerings och test data. Förklara hur de olika delarna används.

Träningsdata: Första arbetsmomentet, under vilken modellen lär sig, genom att analysera träningsdata och försöker hitta lämpliga mönster, och skapar vad den upplever som den bästa modellen för uppgiften, normalt 70 %

Valideringsdata: I nästa steg utvärderar man de olika modellerna och väljer man ut den modellen som passar bäst. Därefter ett mellansteg där man kör en ny träningsprocess för den utvalda modellen från tidigare Training och valideringsdata, normalt 15 %

Testdata: I det sista steget kontrollerar man hur bra modellen fungerar, beräkningen ger ett estimat error, momentet ger helt data som modellen har någon erfarenheten av tidigare så detta steg påvisar hur väl modellen kan generalisera. Normalt 15 % osedd data för utvärdering.

5) Förklara vad koden gör

```
n_cols = x_train.shape[1]
```

Koden ger antal kolumner i modellen, där input kommer från `shape[1]` av datasetet, `X_train`, `shape[0]` ger antal rader, och `shape[1]` ger antal kolumner

```
nn_model = Sequential()
```

Med hjälp av Keras skapas ett tomt sequential – objekt, vilket är utgångspunkten, en behållare, för ett neuralt nätverk vid namn nn-model

```
nn_model.add(Dense(100, activation='relu', input_shape=(n_cols, )))
```

Dolt lager 1: aktivering ReLu, för icke linjära samband

Koden lägger det neurala nätverket `nn_model`, ett dolt lager med 100 neuroner, för att hitta icke linjära samband, används aktiveringsfunktionen, ReLu, input inmatningstorleken hämtas från `n_cols`, som beskrivs ovan.

Dolt lager 2: Regularisering för att minska “overfitting”

```
nn_model.add(Dropout(rate=0.2))
```

Koden skapar ett dolt lager, och där utförs regularisering av modellen, med hjälp av dropout - metoden, med $p = 0.2$, Under träningsfasen inaktivas slumpvis 20 % av neuronerna.

Dolt lager 3 Aktivering ReLu, icke linjära samband, detektion

Ytterligare ett dolt lager med 50 neuroner, och för att detektera icke linjära samband, används återigen ReLu, som matematisk funktion för beräkningen

Dolt lager: Output

Ett lager bestående av 1 neuron, aktiveringsfunktionen sigmoid, mellan 0 – 1, vilket passar bra för binära sannolikheter.

```
nn_model.add(Dense(1, activation='sigmoid'))
```

Koden nedan kompilerar modellen, anger vilken optimeringsfunktion som ska användas, förlust funktionen ska beräknas, samt hur utvärderingen ska göras under träning och utvärdering,

```
nn_model.compile(
```

```
optimizer='adam',
```

Här används Adam (**Adaptive Moment Estimation**) optimering, en vanlig metod inom deep learning.

```
loss='binary_crossentropy',
```

Används för binär klassificering

```
metrics=['accuracy' ])
```

 hur väl man förutsatt rätt klass med hjälp av funktionen accuracy

Modellen nu redo för att tränas

```
early_stopping_monitor = EarlyStopping(patience=5)
```

Keras klass Early stopping, och det fungera enligt följande: Om inte modellens utvärdringsmått accuracy inte förbättrats efter 5 epoch körningar, avslutas program.

```
nn_model.fit( x_train, y_train,
```

Träning av modellen, dataset träningsdata

```
validation_split=0.2
```

Koden avser ta de sista 20 % av datasetet till valideringsdata innan omfördelning av datat.

```
epochs=100,
```

Antal gånger som modellen går och igenom och analysera datamaterial.

```
callbacks=[early_stopping_monitor])
```

Det är ganska vanligt att träning av modellen kan ta ganska lång tid, och för att inte förlora data från fit metoden, används callbacks, och här finns då informationen från hela funktionen early stopping monitor där fit metoden ingår, tillgänglig.

Koden skapar ett neuralt nätverk, n_model, initiering, kompilering och modellen kan efter träning, klassificera binära utfall, baserat på mönster från träningen.

6) Regularisering görs med olika metoder för att reducera “overfitting” av maskinlearning modeller med hög lärande kapacitet, vilket förbättrar modellens generaliseringsförmåga till ny data, och minskar modellens varians,

7) Dropout, en av de mest använda regulariseringsmetoder för deep neural networks, och introducerades 2012–2014.

Dropout principen

Under träningsfasen, sätts en faktor p, dropout rate, som ger sannolikheten att neuroner tillfälligt plockas bort från processen, och kommer heller inte att användas igen under hela träningsprocessen. Nivå på p varierar 10–50 %, beroende på typ av nätverk.

Det gör dropout kraftfullt för att förbättra generaliseringen av ett nätverk. Genom att tvinga modellen att bli mindre beroende av enskilda neuroner och istället förlita sig på mer distribuerad information, minskar risken för överanpassning. Emellertid, med neuron omstruktureringen, medför väsentligt längre tid för modellen att konvergera till ett resultat under träning, men är enligt kursboken värt den extra tiden och ansträngningen, en rimlig trade-off för en i slutändan, bättre och stabilare modell.

8) Early stopping tekniken, där träningen av modellen stoppas automatiskt om den, efter ett visst antal träningscykler (epochs), inte längre förbättrar sitt resultat, loss/. Det innebär att modellen anses ha konvergerat till sitt bästa värde, och träningen avslutas för att undvika överträning. Detta sker när valideringsfelet når sitt minimum, vilket anges i koden, med argumentet, patience, antal konsekutiva epochs.

9) Strukturellt, är bilder spatialt organiserade mönster, och hanteringen av det, är det väsentligt att modellen har god förmåga att extrahera spatiala relationer. I dagsläget är den vanligaste modellen för bildbehandling CNN, Convolutional Neural Network, som använder olika konvolutionslager för att identifiera mönster, och egenskaper. Här används också olika pooling-lager för att reducera dimensionerna på indata, vilket också är en fördel vid bildbehandling. Metoden bakom CNN har sitt ursprung i studier av hjärnans syncentrum – med insikter även från katters synsystem – och har använts för bildbehandling sedan 1980-talet.

10) Convolutional neural networks (CNN, även ConvNet) en arkitektur för deep learning, och är i huvudsak användbar för bildanalys och bildbehandling, är effektiv på att upptäcka och känna igen objekt, mönster i bilder och göra olika kategoriseringar, kan även användas inom signal processen, arbete med tidserier samt för ljud data. CNN är lämpligt att använda för stora modeller med komplexa datasystem

Användningsområden

Medicinska bilder

Audio hantering

Objekt detektion

Syntetisk data generering: Kan med hjälp av GANs skapa nya bilder som kan användas för ansiktedetektion, emotional detektion samt med självkörande fordon.

11) Vad gör nedanstående kod

```
model.save("model_file.keras")
```

När modellen är färdig, kan den sparas med funktionen `model_save ()` under namnet `model_file_keras`

```
my_model = load_model("model_file.keras")
```

När modellen ska användas, ges modellen namnet `my_model` och den sparade modellen laddas in med funktionen `load_model`.

12) CPU, central processing unit, en hårdvara, processen som har uppgift att tolka och exekvera hårda och mjukvaror instruktioner, samt utföra samtliga beräkningar och logik.

En GPU (graphics processing unit), ursprungligen utvecklad för grafikhantering, bilder, animering och videos, men har numera bredare användningsområden även inom dataanalys och machine inlärning.

En CPU har normalt kanske 4–5 processkärnor och arbetar sekventiellt, medan en GPU kör ett antal parallella processorer, och dela upp arbetet i en stor mängd undergrupper.

System med GPUs, på grund av beräkningskraft, hastighet och parallell hanteringen, är därför idealisk för datakörningar inom maskininlärning.

Självutvärdering

1) Roligast med kunskapskontrollen

Lite oväntat, tycker jag det var rätt intressant att gå igenom teoriuppgiftern kring koden för Deep learning och vad händer i varje programsteg och tänka igenom och beskriva koden, steg för steg.

Design och python kodning av en RAG-chatbot, intressant att konkret se chatbotens respons, och utveckling steg efter steg utveckla programmet. Ursprungligen hade jag tanken på en matematik chatbot, men insåg snabbt att här behövs istället en mer kommunicerade bot utifrån given context.

Och Väjer då att skapa en juridisk chatbot, som göra nytta, på ett lättsamt sätt och enkelt sätt diskutera skatterättsliga frågor, kanske finns intresse av den typen av support och kunskaps-baserad chatbot.

Testkörningar med Gemini API: se hur den svarar, justerade olika parametrar

Interaktion mellan retrieval och generering – Det var intressant att få systemet att förstå kontext och hämta rätt juridisk information innan det genererade svar.

Testning och evaluering – Att använda DeepEval för att mäta chatbotens prestanda gav konkret feedback och möjligheten att förbättra svarens kvalitet.

Direkt programrespons – Den mest engagerande delen var att se systemets återkoppling och hur chatboten presterade i verkliga scenarion.

2) Betyg

- ✓ Besvarat på 12 teoretiska frågor
- ✓ Byggt Rag Chatbot
- ✓ Utvärdering RAG chatbot med Deep Eval
- ✓ Analysera framtida möjligheter, problem, styrkor, översiktlig SWOT analys på Chatboten

3) Mest utmanade och lösningarna

Från regeringens hemsida, lite problem med API körningen, för att just få nerladdning av inkomstskattelagen, tänkte alternativ 1 : web scraping alternativ 2: kan ju faktiskt bara spara helt enkelt, en pdf fil från lagen fulltext.

I samband med körning av embedding efter chunking stötter på lite problem med den förvalda modellen, Megatron Bert,

Problem

4) En skattrådgivande chatbot, i verkliga tillämpning

SWOT analys

Potentiella utmaningar

Juridik har uppskattningsvis senaste 200 år, varit en personbasad profession, med arbetsverktygen papper, penna och lagbok. Området på grund av sin unika ställning har fått utrymme att färdigställa resultat enligt självständig tidsplan, på grund av den efterfrågade kompetensen, vilket lett till vad många anser orimligt hög timdebitering,

Även år 2025, präglas branschen av långa handläggningstider i domstolar på samtliga nivåer, ända upp till Högsta domstolen. Med den bakgrunden, är tendensen att betrakta juridisk information från en AI, som opålitlig eller som en otänkbar tanke.

Därtill är juridikbranschen tydligt hierarki som ofta betraktar innovationer som en riskfylld process att värja sig ifrån, och i bästa fall helt undvika.

Icke korrekt information: Scenariot att chatboten ger missvisade information eller feltolkningar, och att användare anser att man blivit utlovad eller ännu värre garanterad en viss utgång i en juridisk fråga.

Juridiska aspekter

Chatboten är konfigurerad att hantera svenska Inkomstskattelagen, i den första versionen, på de lagar som rör privatpersoner. Chatbotens svar är baserat på en direkt och korrekt rättskälla.

Vid användning, i kommersiella verksamhet, bör en friskrivning klausul inkluderas, boten kan göra misstag, och inget tekniskt system kan garantera 100 % tillförlitlighet.

Om användaren är införstådd och godkänner villkoren innan chatten påbörjas. Chatboten gör sitt bästa för att besvara frågorna korrekt och effektivt.

Affärsmässigt och Marknadsföring

Konkret värde – Hur chatboten löser ett verkligt problem och effektiviserar frågor inom skatterätt.

Konkurrensfördelar och USP, Hur unik lösning är chatboten jämfört med andra tjänster och traditionella rådgivare.

Affärsmodell – Ska chatboten vara gratis, prenumerationsbaserad eller en betaltjänst?

Tillväxtpotential – Kan den utvecklas för fler juridiska områden eller integreras med andra system, tex ekonomi och logistik, bank och aktiedepå API mot Skatteverket.

Målgruppsanalys – Vem är den primära användaren? Privatpersoner, småföretagare eller större aktörer?

På den svenska marknaden idag finns åtminstone 3 chatbotar med juridiskt fokus, Juno, Lexly och Lagen.law.

Etiska aspekter

Hur säkerställer chatboten transparens och användarkontroll över den information som genereras? I huvudsak, är det inte tänkt att chatboten ska spara information om användaren, men för framtida bruk kan det bli frågan om inkluderade av en GDPR policy och även EU:s AI act

Tekniska risker

Risk för att chatboten blir utsatt för olika dataattacker och hacking försök.

Vanliga säkerhetsrisker

Prompt Injection – inmatning av osynliga kommandon och på annat sätt försöka ändra chatboten.

DDoS-attacker – Om systemet är webbaserat kan det utsättas för överbelastningsattacker som gör tjänsten otillgänglig.

Gemini API-Exploit – risk för svaghet i integrationen däremellan exploateras.

Man-in-the-Middle (MITM) Om dataöverföringen inte är krypterad, kan någon avlyssna kommunikationen mellan användare och boten

En säkerhetsanalys behöver göras då innan chatboten skulle tas i bruk, alla chatmeddelande sker via krypterade data, med hjälp över SSL/TLS.

Input-skydd – Se till att chatboten filtrerar och sanerar användarinmatningar för att undvika manipulation.

Begränsa antalet frågor per användare för att förhindra överbelastningsattacker.

Loggning & övervakning: Skapa system för att detektera ovanliga beteenden och flagga dem.